Styled Markdown Example

This is a paragraph with **bold** and *italic* text.

Lists # SELENIUM + TESTNG INTERVIEW MASTER GUIDE (Fully Explained Edition)

Every topic below has 4 parts. Read them in this order:

1. **Understand it** — a simple explanation with a real-life example, so the concept actually makes sense.
2. **Code/Example** — what it looks like when written.
3. **MEMORY HOOK** — a short phrase to lock it in your brain.
4. **SAY THIS** — what to speak in the interview, in 20–40 seconds.

SECTION 1 — "EXPLAIN YOUR FRAMEWORK" (asked in EVERY interview)

1.1 Understand the framework first (then the answer becomes easy)

Think of a framework like a **restaurant**:

- The **waiter** (your test script) never cooks — he just takes the order to the kitchen.
- The **kitchen** (your page classes) does the actual work.
- The **recipe book** (your config/Excel files) holds the ingredients, so you never run to the market for each order.
- The **manager** (TestNG) decides which orders come first, which waiter serves which table, and checks if the food was good.
- The **bill and photos of dishes** (Extent reports + screenshots) prove what happened.

A "framework" is nothing mysterious — it is just a folder structure + rules that keep your automation **organized, reusable, and easy to fix when the application changes.**

1.2 The 60-second script (memorize word-for-word)

> "Our framework is a **Maven-based Hybrid framework** using **Java, Selenium WebDriver, TestNG and the Page Object Model**.
> It has six blocks:
>
> 1. **Config block** — properties files and Excel hold the URL, browser, credentials and test data, so nothing is hardcoded in the code.
> 2. **Base block** — a BaseTest class that opens the browser before every test and closes it after.
> 3. **Pages block** — every webpage is one Java class holding its locators and actions. This is the Page Object Model.
> 4. **Tests block** — TestNG test classes that only call the page methods and check the results. No locators here.
> 5. **Utils block** — reusable helpers: waits, screenshots, Excel reader.
> 6. **Reports block** — Extent Reports with screenshots on failure, plus Log4j logs.
>
> The whole suite runs from a **testng.xml**, which lets me group tests and run them in parallel. The code lives in **Git**, is built by **Maven**, and runs nightly on **Jenkins**, which publishes the report and emails the team."

1.3 MEMORY HOOK for the six blocks

"Can Big Projects Test Until Reports Come?"

Word	Block	What lives there
Can → Config	config/	.properties + Excel (data, not code)
Big → Base	base/	BaseTest — open/close browser
Projects → Pages	pages/	one class per webpage
Test → Tests	tests/	TestNG classes + assertions
Until → Utils	utils/	waits, screenshots, Excel reader
Reports → Reports	reports/	Extent HTML, logs, screenshots
Come → CI	Jenkins	mvn clean test runs nightly

If you can say that one sentence, you can draw the whole framework on the whiteboard without missing a block.

1.4 The project tree (be able to draw this)

```
SeleniumTestFramework/
├── pom.xml                          ← all JARs (dependencies) declared here
├── testng.xml                       ← which tests to run, parallel or not
└── src/
    ├── main/java/                               ← THE FRAMEWORK (reusable
library)
    │     ├── base/BaseTest.java                 → opens/closes browser
    │     ├── pages/LoginPage.java               → locators + login() method
    │     ├── pages/HomePage.java                → locators + actions
    │     └── utils/                             → WaitUtils, ScreenshotUtils,
ExcelUtils
    └── test/
          ├── java/
          │     ├── tests/LoginTest.java              → @Test methods +
assertions
          │     └── listeners/TestListeners.java    → screenshot on
failure
          └── resources/
                ├── config.properties      → url, browser, username,
timeouts
                ├── testdata/ExcelData.xlsx → data rows for DataProviders
                └── log4j2.xml             → logging configuration
```

Q: What is the difference between src/main/java and src/test/java? (asked when they see your tree)

Understand it: Maven forces this split on purpose —

- **src/main/java** = your **framework / library code** — Base, Pages, Utils. It gets compiled, and with `mvn install` it becomes a **JAR** that other projects can also reuse.
- **src/test/java** = the **tests that USE the framework** — your TestNG classes and listeners. It can see everything in src/main/java, but NOT the other way around.
- **src/test/resources** = test data and configs (Excel, .properties, log4j2.xml) — automatically on the test classpath, so `config.properties` and Excel paths are found at runtime.

Two conventions you should be able to talk about:

1. **Reusable-framework style (what the tree above shows):** base/pages/utils in `src/main/java`, tests in `src/test/java`. Advantage — `mvn install` builds a framework JAR that 3 different

product test-suites can depend on; the framework is versioned and shared like a product.

2. **Small-project style:** everything under `src/test/java`. Totally valid for a single-suite project — many teams do it. If asked "why is everything in test?" — "the framework is used only by this one suite, so we keep it simple."

Bonus line that earns points: "Dependencies with `<scope>test</scope>` — like TestNG — are visible ONLY inside src/test/java; that's why the scope matters."

MEMORY HOOK: *"Main is the machine, Test is the test drive"* — main builds the library, test uses it.

SAY THIS: "We keep the framework — base, pages, utils — in src/main/java so it compiles as a reusable library, and the TestNG tests and listeners in src/test/java, with test data and config in src/test/resources. That way the same framework can serve multiple test suites, and test-scoped dependencies like TestNG stay isolated to the test side."

Why this structure is powerful (say this if asked "why so much structure?"):
"If the login button's ID changes tomorrow, I change it in ONE file — LoginPage.java — and all 50 tests that use login keep working. Without the framework, I would have to find and fix the ID inside 50 different test scripts."

1.5 Why Page Object Model? (asked immediately after the framework question)

Understand it: Imagine you write `driver.findElement(By.id("login")).click()` in 30 test scripts. One day the developer changes the button. Now you must edit 30 files, and you will surely miss a few. POM says: keep the locator in ONE class (the page class). Tests never touch locators directly — they only call methods like `loginPage.login(user, pass)`.

Three benefits, in plain words:

- **One place to fix** (maintainability)
- **Tests read like English** (readability): `homePage.verifyWelcomeMessage()` instead of 10 lines of locator code
- **Reuse** (the same page class serves 50 tests)

MEMORY HOOK: *"Locators live with their page; tests stay clean."*

SECTION 1.6 — THE END-TO-END FLOW OF ONE TEST RUN (XML → REPORT, INCLUDING TEST

DATA)

Understand it: This is the question "walk me through the flow of your framework from start to end" — the full journey of ONE suite run, from Jenkins firing the command to the team receiving the report. Learn the 15 steps as a story:

```
Jenkins (nightly, or you running mvn clean test locally)
   |
   ▼  command: mvn clean test
1. MAVEN
   • reads pom.xml
   • downloads JARs (Selenium, TestNG, POI...) from Maven Central → .m2 folder
   • surefire plugin opens testng.xml
   |
   ▼
2. TESTNG ENGINE reads the suite
   • registers LISTENERS (from the XML)
   • creates objects of the test classes
   |
   ▼
3. @BeforeSuite  → ConfigReader loads config.properties (URL, browser, timeouts)
                   • ExtentReports object created (report session starts)
   |
   ▼
4. @BeforeTest   → applies the browser parameter for this <test> block
   |
   ▼
5. @BeforeClass  → WebDriverManager fetches chromedriver • browser opens
                    • page objects created: new LoginPage(driver)
   |
   ▼
6. DataProvider runs BEFORE the test it feeds  ◄── TEST DATA ENTERS HERE
   • ExcelUtils (Apache POI) opens ExcelData.xlsx
   • reads every data row → returns Object[][]
   • the @Test will now run ONCE PER ROW
   |
   ▼
7. @BeforeMethod → fresh login, clean state for each test
```

```
   |
   ▼
8. @Test (row 1)
   • calls page methods: lp.login(user, pass)   ← POM layer
   • pages use WaitUtils (explicit waits)         ← Utils layer
   • Assert.assertEquals(actual, expected)         ← verdict recorded
   |
   ▼
9. LISTENERS fire automatically:
   • PASS → onTestSuccess → green in report
   • FAIL → onTestFailure → SCREENSHOT captured + attached to Extent,
Log4j entry
   |
   ▼
10. @AfterMethod → logout / cleanup
    |
    ▼
11. Loop back to step 7 for row 2, row 3... and every other @Test
    |
    ▼
12. @AfterClass → driver.quit() (browser closes)
13. @AfterTest  → cleanup for the <test> block
14. @AfterSuite → extent.flush()  ← report file WRITTEN to disk
    |
    ▼
15. MAVEN finishes the build → Jenkins publishes the HTML report
    → email/Slack notification to the team
```

The 3 doors where TEST DATA enters (interviewers love this):

Door	File	What it holds	Example
1. Environment data	config.properties	same for ALL tests	URL, browser, timeouts
2. Input data	Excel → DataProvider	many rows, test runs once per row	username/password combos
3. Run-time params	testng.xml `<parameter>`	chosen per run	browser=chrome or firefox

SAY THIS (the 60-second spoken version):

> "Jenkins triggers `mvn clean test`. Maven reads pom.xml, downloads the dependencies, and the surefire plugin loads testng.xml. TestNG registers the listeners and starts the suite. In @BeforeSuite we load config.properties and start the Extent report. @BeforeClass opens the browser and creates the page objects. The DataProvider then reads test data rows from Excel using Apache POI, so each test runs once per row. Inside the @Test we call page object methods with explicit waits and assert the results — if anything fails, the listener's onTestFailure captures a screenshot and attaches it to the report automatically. After all tests, @AfterClass quits the driver and @AfterSuite flushes the Extent report, which Jenkins publishes and emails to the team."

MEMORY HOOK (one line — memorize this):

> *"Jenkins fires Maven, Maven loads TestNG, Config feeds Base, Excel feeds Data, Pages serve Tests, Listeners catch failures, AfterSuite writes the report, Jenkins emails it."*

Short version of the same hook: **Build → Suite → Base → Data → Test → Listener → Report → Email.**

SECTION 2 — FRAMEWORK TYPES

Understand each with one line:

Type	What it means in simple words	Example
Linear	One long recorded script, top to bottom. No functions.	Record & playback
Modular	Break the app into small functions/modules and reuse them.	`login()`, `search()` used everywhere
Data-driven	ONE test, MANY rows of data — run the same steps with different inputs.	Login test runs 5 times with 5 usernames from Excel
Keyword-driven	The test steps themselves are written in Excel as keywords (Click, Enter, Verify) and code reads and executes them.	Row: `Click
Hybrid	A mix — usually POM + data-driven + modular.	Most real company frameworks

SAY THIS: "Ours is a Hybrid framework — POM for structure, plus data-driven testing where TestNG DataProviders read rows from Excel using Apache POI."

MEMORY HOOK: *"L-M-D-K-H: Lazy Monkeys Don't Keep Hybrids"* — Linear, Modular, Data-driven, Keyword, Hybrid.

SECTION 3 — SELENIUM CORE

Q: What is Selenium?

Understand it: Selenium is a free, open-source set of tools that lets a program **control a web browser like a human** — open pages, click buttons, type text, read text. It cannot think; it only does what your code tells it.

Four components:

1. **Selenium IDE** — a browser plugin that records your clicks and replays them. Good only for quick demos. No logic, no loops, no assertions.
2. **Selenium RC** — the OLD, retired tool (Remote Control). It worked by injecting JavaScript into the page. Know it only for history questions.
3. **Selenium WebDriver** — what everyone actually uses today. Your Java code sends commands to a small driver program (chromedriver.exe) which moves the real browser.
4. **Selenium Grid** — run the same tests on many machines/browsers at the same time, in parallel.

MEMORY HOOK: *"I-R-W-G: Ide, RC, WebDriver, Grid — only W works for real work."*

Q: What happens when you write driver.get()? (Architecture)

Understand it step by step:

1. Your Java code (`driver.get("...")`) does NOT touch the browser directly.
2. It converts your command into an **HTTP request** (like a small web message) and sends it to **chromedriver.exe** (or geckodriver for Firefox).
3. The driver receives the request and controls the real Chrome/Firefox browser.
4. The browser does the action and sends back a response.

So the chain is: **Your code → HTTP → driver exe → real browser.**

In Selenium 4 this message format is the **W3C WebDriver protocol** — a worldwide standard, which makes commands behave the same in every browser.

MEMORY HOOK: *"Code → Driver → Browser, always through HTTP."*

Q: Selenium advantages and limitations?

Advantages (say 4): free and open-source; works with Java/Python/C# and all major browsers; big community; fits perfectly with TestNG/Maven/Jenkins.

Limitations (know these — interviewers check honesty):

- Web browsers only — cannot automate desktop apps (that's WinAppDriver) or mobile apps (that's Appium).
- Cannot read/handle CAPTCHA or OTP by design — these exist to stop bots.
- No built-in reports or test management — that's why we add TestNG, Extent, etc.

MEMORY HOOK: *"Web only, no captcha, brings no reports of its own."*

SECTION 4 — LOCATORS & XPATH (asked in every single interview)

Q: How many locators? Which one is best?

Understand it: To click or type anywhere, Selenium first needs an "address" of that element in the HTML. There are 8 ways to give that address:

`id`, `name`, `className`, `tagName`, `linkText`, `partialLinkText`, `cssSelector`, `xpath`

Which is best and WHY (this is what interviewers want):

- **id** is best because a good developer makes it unique on the page — fast and exact, like an Aadhaar number.
- **name** is second — usually unique too.
- **CSS** — good for combinations of tag + attribute.
- **XPath** — most powerful (can search text, go up to parents) but slowest — the last resort.

Order to say: id → name → className → CSS → XPath.

MEMORY HOOK: *"ID is the King; XPath is the last resort."*

Q: Difference between CSS and XPath?

Understand with an example. Suppose a button:

```
<button id="submit" class="btn">Save</button>
```

- CSS: `button#submit` or `button.btn`
- XPath: `//button[@id='submit']`

The big practical difference: **XPath can search UPWARD (find a parent), CSS cannot.** XPath can also match visible text (`//button[text()='Save']`) — CSS cannot match text at all.
If you need "the row that contains the word Laptop, then its price cell" — you need XPath axes.

MEMORY HOOK: *"CSS goes down only; XPath goes up and down."*

Q: Absolute vs Relative XPath?

Absolute: starts with a single slash `/` , gives the FULL path from the root:
`/html/body/div[2]/div[1]/form/input[1]`
Problem: if ANY tag in the chain moves, the whole path breaks. Like giving your address as "Earth → India → 3rd building → 2nd floor → left door".

Relative: starts with double slash `//` , searches anywhere in the page:
`//input[@id='username']`
This is what you should always use.

MEMORY HOOK: *"One slash = fragile; two slashes = flexible."*

Q: XPath functions for DYNAMIC elements (very common)

The problem: some sites generate ids like `username_83721` — the number changes every page load, so `@id='username_83721'` fails next time.

The fix — three tools:

```
// contains() — matches any id that CONTAINS the word 'username'
//input[contains(@id,'username')]

// starts-with() — matches ids that BEGIN with 'username_'
//input[starts-with(@id,'username_')]

// text() — match by visible text
```

```
//button[text()='Login']
//a[contains(text(),'Forgot Password')]
```

MEMORY HOOK: *"ID keeps changing? contains() it."*

Q: XPath axes (the "find the neighbour" trick)

Understand it: Axes mean "from this element, now look at its relatives."

Classic interview table question — "How do you find the price of a product when you only know its name?"

```
<tr>
  <td>Laptop</td>      <!-- name -->
  <td>45000</td>       <!-- price -->
</tr>


// find the cell containing 'Laptop', then the NEXT cell (sibling) in the
//td[text()='Laptop']//following-sibling::td[1]
```

Other axes: `parent`, `ancestor`, `child`, `preceding-sibling`, `following`, `preceding`.

MEMORY HOOK: *"following = BELOW in the HTML, preceding = ABOVE."*

Q: findElement vs findElements?

Understand it:

- `findElement` → "give me ONE element" → returns the FIRST match. If nothing matches → **throws NoSuchElementException** (test crashes).
- `findElements` → "give me ALL matching elements" → returns a List. If nothing matches → **returns an EMPTY list, no error** — like searching your contacts for "Xyz": one search shows an error popup, the other just shows an empty list.

Classic use — count all links on a page:

```
List<WebElement> links = driver.findElements(By.tagName("a"));
System.out.println(links.size());
```

MEMORY HOOK: *"One throws, the List lies low (comes back empty)."*

Q: How do you find broken links?

Understand it: Selenium cannot open a URL and check if it is alive — but plain Java can. So:

1. Collect all `<a>` tags (findElements).
2. Read each one's `href` attribute (`getAttribute("href")`).
3. Skip empty and javascript links.
4. Send each URL an HTTP request in Java (HttpURLConnection) and read the **status code**: 200 = fine, 404/400 = broken.
5. Print/report all broken ones.

MEMORY HOOK: *"Status 200 = alive; 4xx = dead."*

SECTION 5 — WAITS (the #1 practical interview topic)

Q: Why do we need waits at all?

Understand it: Selenium runs at computer speed; the page loads at network speed. You ask Selenium to click a button that hasn't appeared yet → crash. Waits = telling Selenium "wait for the page to be ready, not just run blindly." This is called **synchronization**.

Q: Implicit vs Explicit vs Fluent wait?

Implicit wait — the "global blanket":

```
driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
```

You set it once, and it applies to EVERY findElement in the whole script: "if you don't find the element, keep trying for up to 10 seconds before giving up."
Simple, but dumb — it can only wait for the element to EXIST. It cannot wait for a button to become clickable or a loader to disappear.

Explicit wait — the "targeted sniper":

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement btn = wait.until(
    ExpectedConditions.elementToBeClickable(By.id("submit")));
btn.click();
```

You wait for ONE specific condition on ONE specific element: clickable, visible, text present, alert shown, etc. This is what a professional framework mostly uses.

Fluent wait — "a sniper with a custom clock":

```
Wait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))      // max total wait
    .pollingEvery(Duration.ofSeconds(3))      // check every 3 seconds
    .ignoring(NoSuchElementException.class);   // don't crash while polli
```

Same idea as explicit, but you control HOW OFTEN to check and WHICH errors to ignore while checking. Useful for slow-loading elements that throw StaleElement errors.

MEMORY HOOK: *"Implicit = blanket over everything; Explicit = one element, one condition; Fluent = explicit with a timer and patience."*

Q: Why is Thread.sleep(3000) bad?

Understand it: Thread.sleep **always sleeps the full 3 seconds**, even if the page was ready after 200 milliseconds — so the test is unnecessarily slow. And if the page needs 5 seconds? It still crashes. Waits are smart: they keep checking and continue the MOMENT the condition is true.
MEMORY HOOK: *"Sleep is blind waiting; waits are smart waiting."*

Q: presence vs visibility vs clickability? (a favorite trick question)

- **presence** — the element exists in the HTML (the DOM), but might be invisible (hidden).
- **visibility** — it exists AND is actually rendered on screen (you could see it).
- **clickable** — visible AND enabled (a disabled greyed-out button is visible but NOT clickable).

MEMORY HOOK: *"In DOM → on screen → ready to use."*

Q: Can we mix implicit and explicit waits?

Best answer: **No.** When both are set, each polling cycle of the explicit wait ALSO waits the implicit time — the delays stack up and become unpredictable. Selenium's own docs advise against mixing.

MEMORY HOOK: *"Never mix blanket and sniper."*

SECTION 6 — WINDOWS, FRAMES, ALERTS

Q: close() vs quit()?

close() — closes ONLY the current browser window. The session (the connection between your code and the driver) is still alive; other windows stay open.
quit() — closes ALL windows AND ends the session completely. If you forget quit(), chromedriver.exe processes stay running in the machine's memory like zombies.

MEMORY HOOK: *"Close one, Quit ALL + kill the session."*

Q: get() vs navigate()?

`driver.get("url")` — open a page and wait for it to load.
`driver.navigate().to("url")` — opens too, but navigate() also gives you **back(), forward(), refresh()** — the browser's back/forward arrows in code form.

MEMORY HOOK: *"get goes; navigate goes AND can come back."*

Q: How do you handle multiple tabs/windows?

Understand it: Every browser window/tab has an ID (handle). Selenium can only work in ONE window at a time — you must "switch" your focus by giving the ID.

```
String parent = driver.getWindowHandle();      // THIS window's ID (single

driver.findElement(By.id("openNew")).click();  // new tab opens

Set<String> all = driver.getWindowHandles();   // ALL window IDs (a Set)

for (String handle : all) {
    if (!handle.equals(parent)) {
        driver.switchTo().window(handle);        // move focus to the new
```

```
    }
}
// do your work in the new tab...
driver.close();                                  // close the tab
driver.switchTo().window(parent);                // come back home
```

MEMORY HOOK: *"Handle = THIS one (String); HandleS = ALL of them (Set). The 'S' is everything."*

Q: How do you handle iframes?

Understand it: An iframe is a page INSIDE a page (like a TV inside a wall). Selenium can only see the MAIN page. If your element is inside the iframe, Selenium is "standing outside the room" and cannot touch it — until you **switch into** the frame.

```
driver.switchTo().frame(0);              // by index (0 = first frame)
driver.switchTo().frame("frameName");    // by name or id
driver.switchTo().frame(element);        // by the frame WebElement
driver.switchTo().defaultContent();      // step back out to the main pag
```

How do you KNOW an element is in a frame? In DevTools, the element sits under a `<iframe>` tag.

MEMORY HOOK: *"Frames are rooms: switchTo to enter, defaultContent to come home."*

Q: How do you handle JavaScript alerts (OK/Cancel popups)?

Understand it: These are the browser's own grey popups (alert/confirm/prompt). They are NOT part of the HTML — so locators NEVER work on them. You must switch to them first:

```
Alert alert = driver.switchTo().alert();
alert.getText();        // read the message
alert.accept();         // click OK
alert.dismiss();        // click Cancel
alert.sendKeys("hi");   // type into a prompt popup
```

MEMORY HOOK: *"Accept = OK, Dismiss = Cancel — and no locator ever works on alerts."*

Q: What about OS-level popups (like the Windows file dialog)?

That dialog belongs to the operating system, not the browser — Selenium cannot touch it. Use Java's **Robot** class (simulates real keyboard/mouse) or an AutoIt script.
BUT say this first: "For file upload, most sites use `<input type="file">` — then I don't open any dialog at all; I just `element.sendKeys("C:/path/file.png")` . Robot is only the plan B."

MEMORY HOOK: *"Input tag? sendKeys. Real OS dialog? Robot."*

SECTION 7 — ACTIONS, JAVASCRIPT, SCREENSHOTS, DROPDOWNS

Q: Actions class — what is it for?

Understand it: Normal `.click()` and `.sendKeys()` cover simple cases. The Actions class handles **mouse gestures**: hover, right-click, double-click, drag and drop.

```
Actions a = new Actions(driver);
a.moveToElement(menu).perform();            // hover — makes hidden subme
a.contextClick(element).perform();          // right-click
a.doubleClick(element).perform();          // double-click
a.dragAndDrop(source, target).perform();   // drag and drop
a.keyDown(Keys.CONTROL).click(item).keyUp(Keys.CONTROL).perform(); // mul
```

Two things people forget in interviews: (1) every chain must end with `.perform()` or nothing happens; (2) keyboard combos (Ctrl, Shift) also come from Actions + the Keys enum.

MEMORY HOOK: *"Mouse = Actions, and without perform() it's only a plan."*

Q: JavascriptExecutor — when do you actually need it?

Understand it: Sometimes the normal click fails because an ad, spinner, or invisible overlay covers the button ("ElementClickInterceptedException"). Or you need to scroll. Then you ask the browser to run raw JavaScript directly:

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("window.scrollBy(0, 500)");                    // scroll dc
js.executeScript("arguments[0].click();", element);              // force a
js.executeScript("window.scrollTo(0, document.body.scrollHeight)"); // sc
```

Say this too (shows maturity): "I use JS as the LAST option, because a JS click bypasses the real user events — if it passes, it doesn't prove a real user could click it."

MEMORY HOOK: *"When Selenium clicks fail, JS never fails — but keep it as plan B."*

Q: How do you take a screenshot?

```
File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(src, new File("screenshots/loginFail.png"));
```

The cast `(TakesScreenshot) driver` works because the driver class implements that interface. (Full automatic-on-failure flow is in Section 9 with listeners.)

MEMORY HOOK: *"Cast, capture, copy — 3 steps."*

Q: Handling dropdowns — two completely different cases

Case 1 — real `<select>` tag → use the Select class:

```
Select s = new Select(driver.findElement(By.id("country")));
s.selectByVisibleText("India");   // by shown text
s.selectByValue("in");             // by the value attribute
s.selectByIndex(2);                // by position, 0-based
s.getOptions();                    // all options — count/verify them
s.getFirstSelectedOption();         // what is currently chosen
```

Case 2 — "fake" dropdown (div/ul/li — Bootstrap, React) → Select class does NOT work:
click to open it, then find all options with findElements and loop:

```
driver.findElement(By.id("dropdownToggle")).click();
List<WebElement> options = driver.findElements(By.cssSelector("ul.dropdow
for (WebElement o : options) {
```

```
    if (o.getText().equals("India")) { o.click(); break; }
}
```

MEMORY HOOK: *"Select tag → Select class; fake dropdown → click and loop."*

Q: Calendar / date picker?

Two honest answers:

1. If the input field accepts typing — just `sendKeys("24/09/2026")`. Done.
2. If typing is blocked — navigate: click the "next month" arrow until the right month shows, then click the day cell: `//td[text()='24']`.

MEMORY HOOK: *"Type if allowed, else travel month by month."*

Q: Auto-suggest box (like Google search suggestions)?

```
driver.findElement(By.name("q")).sendKeys("selenium");
List<WebElement> suggestions =
    driver.findElements(By.cssSelector("ul li div"));   // wait for them
for (WebElement s : suggestions) {
    if (s.getText().contains("selenium webdriver")) { s.click(); break; }
}
```

Pattern: **type → wait → collect all suggestions → loop → click the match.**

MEMORY HOOK: *"Type, wait, find all, loop, click."*

Q: File upload vs download?

Upload: if the site uses `<input type="file">` — `element.sendKeys("path/to/file.pdf")`. No dialog, no Robot.
Download: tell Chrome WHERE to save (browser options with `download.default_directory`), trigger the download, then verify with Java: `new File("../report.pdf").exists()` inside a small wait loop.

MEMORY HOOK: *"Upload = sendKeys; Download = set the folder + File.exists()."*

SECTION 8 — SELENIUM 4 FEATURES (modern interviews check this)

Understand it: Selenium 4 (2021 onwards) modernized everything around the W3C standard. Five things to know:

1. **W3C WebDriver protocol** — commands are now a worldwide standard, so the same code behaves the same on Chrome/Firefox/Edge. (Before, each browser spoke a slightly different "dialect" — JSON Wire Protocol — causing weird browser-specific bugs.)
2. **Relative locators** — find an element by POSITION instead of attributes: "the input just BELOW the Email label":

```
driver.findElement(RelativeLocator.with(By.tagName("input")).below(emailI
```

Options: `above()`, `below()`, `toLeftOf()`, `toRightOf()`, `near()`.
3. **New Grid** — Grid 4 is easier to run and works with Docker. (Grid 3 = hub + node; Grid 4 = router + distributor + sessions + event bus — just say "distributed architecture".)
4. **Chrome DevTools (CDP) support** — simulate slow network / offline, fake geolocation, read browser console logs — great for testing "what if the user's internet is bad".
5. **New window/tab + Shadow DOM support** — `driver.switchTo().newWindow(WindowType.TAB)`, and `element.getShadowRoot()` to reach elements inside shadow DOM (which XPath can't otherwise touch).

MEMORY HOOK: *"W-R-N-C-S: Why Really New Cool Stuff"* → W3C, Relative locators, New Grid, CDP, Shadow DOM.

SECTION 9 — TESTNG (half of every interview)

Q: What is TestNG? Why not JUnit?

Understand it: TestNG = "Test Next Generation". It is a framework that **manages your tests** — which run first, what data they use, what happens before/after each test, what counts as pass/fail, and what report comes out.
Why it beats old JUnit in one line: JUnit ran tests; TestNG **orchestrates** them — grouping, ordering, dependencies, parallel runs, data files, retries.

MEMORY HOOK: *"NG = Next Generation — the generation after JUnit."*

Q: What does TestNG provide to Selenium? (VERY common question)

Understand it: Selenium by itself is only a **remote control for the browser** — it can open, click, type, read. It has NO idea what a "test" is. TestNG is the brain on top:

Selenium alone cannot...	TestNG gives you
mark "this method is a test"	`@Test` annotation
do setup before / cleanup after	`@BeforeMethod`, `@AfterMethod`, `@BeforeSuite` ...
check expected vs actual	`Assert`, `SoftAssert`
control order/dependencies	`priority`, `dependsOnMethods`, `groups`
run in parallel	`parallel` + `thread-count`
feed data rows	`@DataProvider`, `@Parameters`
auto-retry flaky tests	`IRetryAnalyzer`
produce a report	built-in HTML + Extent via listeners
run selected combinations	`testng.xml`

MEMORY HOOK: *"Selenium DRIVES, TestNG DECIDES."* (drives the browser vs decides what/when/how and judges pass-fail.)

SAY THIS: "Selenium only automates browser actions — it cannot organize, assert, or report. TestNG adds the test layer: @Test annotations, before/after hooks, assertions, grouping, priorities, DataProviders, parallel execution, listeners and retry logic."

Q: Write the TestNG annotation hierarchy (THE most asked TestNG question)

```
@BeforeSuite → @BeforeTest → @BeforeClass → @BeforeMethod → @Test
→ @AfterMethod → @AfterClass → @AfterTest → @AfterSuite
```

Understand it with the hotel analogy:

- **Suite** = the whole hotel (all testng.xml suites) — biggest scope
- **Test** = one floor (a `<test>` block in the XML)
- **Class** = one room (a test class)
- **Method** = the bed — the smallest, most personal space (one @Test method)

Setup runs **biggest → smallest** (open the hotel before you make the bed).
Teardown runs **smallest → biggest** (leave the bed before locking the hotel).

Practical mapping interviewers like:

- `@BeforeSuite` — start the Extent report, load config.properties (once for everything)
- `@BeforeTest` — set the browser for this `<test>` block
- `@BeforeClass` — open browser once for the whole class
- `@BeforeMethod` — fresh login before EVERY test (clean state each time)
- `@AfterMethod` — logout / screenshot on failure
- `@AfterSuite` — flush (write) the report

MEMORY HOOK: *"Biggest→smallest for setup, smallest→biggest for teardown."*

Q: If no priority is given, in what order do tests run?

Alphabetically by method name (a, b, c...). Fix it with:

- `@Test(priority = 1)` — lowest number runs first; negatives allowed; default 0
- `@Test(dependsOnMethods = "createOrder")` — run only after that method PASSES; if it fails, this test is SKIPPED (shows as skipped, not failed — good talking point)
- `alwaysRun = true` — run even if the depended-on method failed
- `enabled = false` — switch a test off without deleting it
- `invocationCount = 5` — run the same test 5 times (for stability checks)
- `timeOut = 5000` — fail the test if it exceeds 5 seconds
- `expectedExceptions = ArithmeticException.class` — pass only if that exception occurs

MEMORY HOOK: *"No priority = A before B before C."*

Q: What is testng.xml and why is it useful?

Understand it: testng.xml is a control panel for your suite — you choose which tests, which groups, which browser, how many threads — WITHOUT touching Java code. A manual tester can also edit it.

```
<suite name="Regression" parallel="tests" thread-count="3">
  <listeners>
    <listener class-name="listeners.TestListeners"/>
  </listeners>
  <test name="Smoke">
    <parameter name="browser" value="chrome"/>
    <groups><run><include name="smoke"/></run></groups>
    <classes><class name="tests.LoginTest"/></classes>
  </test>
</suite>
```

MEMORY HOOK: *"testng.xml is the remote control of the suite."*

Q: Hard assert vs Soft assert (VERY common)?

Understand it:

- **Hard assert (** `Assert` **)** — a strict examiner: first wrong answer → stop the test immediately. Later checks never run, so you see only ONE error.
- **Soft assert (** `SoftAssert` **)** — a patient examiner: notes every wrong answer and lets the test continue; at the end, `assertAll()` reports ALL failures together.

```
SoftAssert sa = new SoftAssert();
sa.assertEquals(actualTitle, "Dashboard");     // even if this fails...
sa.assertTrue(welcomeMsg.contains("Welcome")); // ...this still runs
sa.assertAll();   // ← without this line, soft assert failures are silent
```

When to use which: verifying 10 form-field error messages → SoftAssert (want all 10); verifying login landed on Dashboard before acting → hard Assert (no point continuing on the wrong page).

MEMORY HOOK: *"Hard = halt. Soft = collect all, spill at assertAll."*

Q: @Parameters vs @DataProvider?

- **@Parameters** — takes a value FROM testng.xml (`<parameter name="browser" value="chrome"/>`). ONE value per run. Perfect for environment/browser selection.

```
@Test @Parameters("browser")
public void test(String browser) { ... }
```

- **@DataProvider** — a method in Java that returns `Object[][]` (a table of rows). The test runs ONCE PER ROW. Perfect for many data sets.

```
@DataProvider(name = "loginData")
public Object[][] getData() {
    return new Object[][] {
        {"admin", "admin123", true},
        {"admin", "wrongpw",  false}
    };
}
@Test(dataProvider = "loginData")
public void loginTest(String user, String pass, boolean expected) { ... }
```

2 rows = 2 test runs. In real frameworks the Object[ ][ ] is usually filled by reading Excel with Apache POI.

- **@Factory** — creates multiple instances of the whole test class with different data. (Just know the name and one line.)

MEMORY HOOK: *"Parameter = one value from XML; DataProvider = many rows, one run each."*

Q: What are TestNG listeners?

Understand it: A listener is like a **security camera in the exam hall** — it doesn't take the exam, it just WATCHES every event (test started, passed, failed, skipped) and reacts. Your test code doesn't need to know it's being watched.

The important ones:

- **ITestListener** — onTestStart / onTestSuccess / **onTestFailure** / onTestSkipped → here we take screenshots and write to the report
- **ITestResult** — the object carrying which test, what status, what exception
- **IRetryAnalyzer** — retry a failed test automatically (flaky test protection): return `true` from `retry()` to run it again, up to a max count
- **IAnnotationTransformer, ISuiteListener, IExecutionListener** — advanced; just recognize the names

Q: How EXACTLY do listeners take a screenshot when a test fails? (they will ask you to explain the flow)

The 4-step flow:

1. The **BaseTest** class keeps the driver in a `public static` field so any class can reach it.
2. A class `TestListeners` implements **ITestListener** and overrides `onTestFailure(ITestResult result)` .
3. When any test fails, TestNG automatically calls that method. Inside it: take the driver → cast to `TakesScreenshot` → capture → save with the test's name.
4. Register the listener once (in testng.xml) → **every** failing test gets a screenshot automatically, and the tests themselves contain ZERO screenshot code.

```
public class BaseTest {
    public static WebDriver driver;

    @BeforeMethod
    public void setUp() {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }
    @AfterMethod
    public void tearDown() { if (driver != null) driver.quit(); }
}

public class TestListeners implements ITestListener {

    @Override
    public void onTestFailure(ITestResult result) {
        String testName = result.getName();              // which test fa
        TakesScreenshot ts = (TakesScreenshot) BaseTest.driver;
        File src = ts.getScreenshotAs(OutputType.FILE);
        File dest = new File("screenshots/" + testName + "_"
                             + System.currentTimeMillis() + ".png");
        try {
            FileUtils.copyFile(src, dest);
            // attach to Extent report as well:
            // ExtentManager.getTest().addScreenCaptureFromPath(dest.getP
        } catch (IOException e) { e.printStackTrace(); }
    }
}
```

Register it (either way):

```
@Listeners(TestListeners.class)     // on top of the test class
```

```
<listeners><listener class-name="listeners.TestListeners"/></listeners>
```

Why this design is GOOD (say this): "The test classes have no screenshot logic at all — separation of concerns. If we change the report tool tomorrow, we change only the listener, not 200 tests."

MEMORY HOOK: *"Base holds the driver, the Listener hears the failure, the Screenshot is the proof."*

Q: How do you run tests in parallel?

Understand it: Add two attributes in testng.xml: `parallel` (what to split — tests/classes/methods) and `thread-count` (how many run at once).
Danger interviewers love: **each thread needs its OWN driver**, or tests grab each other's browser. Fix: keep the driver in a `ThreadLocal<WebDriver>` in BaseTest.

MEMORY HOOK: *"parallel + thread-count, and keep each driver ThreadLocal."*

Q: What is a flaky test and how do you fix it?

Understand it: A test that passes today, fails tomorrow, passes again — with NO code change. Causes: timing (element not ready), shared data clashing between tests, or a slow environment.
Fixes (say in this order): proper explicit waits → isolated test data → retry with IRetryAnalyzer → then investigate the real cause; retry hides symptoms, it doesn't cure them.

MEMORY HOOK: *"Flaky = random failure; fix = better waits + retry + isolation."*

SECTION 10 — PAGE OBJECT MODEL IN DETAIL

Q: Show the full pattern in code.

```
public class LoginPage {
    WebDriver driver;

    @FindBy(id = "username")    WebElement txtUser;
    @FindBy(id = "password")    WebElement txtPass;
    @FindBy(css = "button[type='submit']") WebElement btnLogin;

    public LoginPage(WebDriver driver) {
```

```
        this.driver = driver;
        PageFactory.initElements(driver, this);    // connects @FindBy fie
    }

    public void login(String user, String pass) {
        txtUser.sendKeys(user);
        txtPass.sendKeys(pass);
        btnLogin.click();
    }
}

public class LoginTest extends BaseTest {
    @Test
    public void validLoginTest() {
        LoginPage lp = new LoginPage(driver);
        lp.login("admin", "admin123");
        Assert.assertTrue(driver.getTitle().contains("Dashboard"));
    }
}
```

Read the test class — it reads like a story, no locators anywhere. That's the whole point.

Q: PageFactory details interviewers check

- `PageFactory.initElements(driver, this)` in the constructor — the ritual. Elements are found **lazily** — not at page load, but the moment you first use each one.
- `@CacheLookup` — caches the element after first find → faster, but breaks if the DOM re-renders → use only for truly static elements.
- **Plain `driver.findElement(By...)` inside page classes is ALSO valid POM.** POM is the IDEA (one class per page), PageFactory is just one way to write it. Say this — it shows depth.

MEMORY HOOK: *"POM is the idea; PageFactory is one way to write it."*

Q: Senior-level extra: page components

Repeated widgets (header, footer, menu) become their own small classes (HeaderComponent) and page classes COMPOSE them. Shows you've worked on real frameworks.

SECTION 11 — COMMON EXCEPTIONS (memorize this table)

Exception	What it really means (plain words)	Typical fix
NoSuchElementException	"I searched, the element doesn't exist" — bad locator or page not loaded	fix locator / wait for visibility
StaleElementReferenceException	"I FOUND it, but the page re-rendered and my reference died" (AJAX refresh)	re-find the element; FluentWait ignoring Stale
ElementClickInterceptedException	"I tried to click but something else (spinner/ad/overlay) is on top"	wait for the overlay to vanish; JS click as plan B
ElementNotInteractableException	element exists but hidden or off-screen	scroll into view; wait for clickable
NoAlertPresentException	switched to an alert that isn't there	wait `alertIsPresent()` first
NoSuchFrameException / NoSuchWindowException	switched to a frame/window ID that doesn't exist	verify the handle exists first
TimeoutException	explicit wait ran out, condition never became true	longer wait or fix the condition
SessionNotFoundException	using the driver after quit()	create a new driver
InvalidSelectorException	your XPath/CSS has a syntax error	fix the syntax
WebDriverException	the parent of all the above	read the message

The classic trap — NoSuch vs Stale:

- NoSuch = element was **never born** (locator wrong).
- Stale = element was found, then the DOM refreshed and the old reference became a **dead link** — like having a friend's old phone number after he changed it.

MEMORY HOOK: *"NoSuch = never born; Stale = born but the page killed it."*

SECTION 12 — SCENARIO QUESTIONS CORNER

- **Dynamic element (ID changes every load)?** Match the stable part: `contains()` / `starts-with()` in XPath, or CSS operators `^=` (starts with), `$=` (ends with), `*=` (contains).
- **CAPTCHA?** Honest answer: "It is designed to stop bots, so we don't automate it. In test environments, developers give a bypass; otherwise I pause for manual intervention." Interviewers LOVE this answer.
- **OTP/2FA?** Read it from an API, database, or a test email inbox (RestAssured/JavaMail), or a fixed test-mode OTP.
- **Shadow DOM?** Elements hidden inside a shadow root — normal locators fail. Selenium 4: `element.getShadowRoot()` ; inside the shadow root only CSS works (no XPath).
- **SVG icons?** No id, normal XPath fails; use `//*[name()='svg']` .
- **Table — read row 3, column 2?** `//table[@id='emp']//tr[3]/td[2]` ; or by content: `//td[text()='Laptop']//following-sibling::td[1]` .
- **Infinite scroll?** Loop: JS-scroll to bottom, wait, until the element count stops increasing.
- **Element visible but not clickable?** Overlay → wait for overlay to disappear → scroll into view → JS click last.
- **Tooltip?** Usually the `title` attribute → `getAttribute("title")` ; CSS-based tooltips need an Actions hover first.
- **getAttribute vs getText?** `getAttribute("value")` = what's written INSIDE the tag's brackets; `getText()` = the visible text BETWEEN the tags. **MEMORY HOOK:** *"Attribute = in the tag; Text = between the tags."*
- **isDisplayed / isEnabled / isSelected?** seen on screen / actually usable (not disabled) / chosen (radio, checkbox).
- **Verify element color?** `getCssValue("color")` .
- **Delete cookies?** `driver.manage().deleteAllCookies()` .
- **Headless mode?** `options.addArguments("--headless=new")` — browser runs with no visible window, for CI servers.
- **Browser from config file?** A factory method in BaseTest reads the `browser` property and returns the matching driver object (`if chrome → new ChromeDriver()`); combined with `@Parameters("browser")` for cross-browser runs.

SECTION 13 — MAVEN + CI/CD (they drill this after the framework)

13.1 The Maven script (memorize — this is exactly what to say)

> "We use **Maven as our build and dependency management tool**. The main configuration file in our project is **pom.xml**.
> In pom.xml we define all the **dependencies** required for automation — Selenium WebDriver, TestNG, reporting libraries, and other utilities.
> Maven **automatically downloads these dependencies from the Maven repository** (Maven Central), so we don't have to manually download JARs and add them to the build path.
> We also use Maven to **compile the project and execute our TestNG test cases** — for example, running the suite with `mvn test`.
> And Maven follows a **standard project structure** — `src/main/java`, `src/test/java` — which keeps the project organized and consistent, so anyone joining the team knows exactly where everything is."

MEMORY HOOK: *"Build + Dependencies + Structure = Maven's 3 jobs."*

13.2 pom.xml dependencies with names + versions (know these numbers)

```
<dependencies>
    <!-- Selenium — browser automation -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.45.0</version>
    </dependency>

    <!-- TestNG — the test framework -->
    <dependency>
        <groupId>org.testng</groupId>
        <artifactId>testng</artifactId>
        <version>7.12.0</version>
```

```
            <scope>test</scope>
        </dependency>

        <!-- WebDriverManager — downloads driver .exe files automatically -->
        <dependency>
            <groupId>io.github.bonigarcia</groupId>
            <artifactId>webdrivermanager</artifactId>
            <version>5.9.2</version>
        </dependency>

        <!-- Extent Reports — HTML reporting with screenshots -->
        <dependency>
            <groupId>com.aventstack</groupId>
            <artifactId>extentreports</artifactId>
            <version>5.1.2</version>
        </dependency>

        <!-- Apache POI — read/write Excel test data -->
        <dependency>
            <groupId>org.apache.poi</groupId>
            <artifactId>poi-ooxml</artifactId>
            <version>5.3.0</version>
        </dependency>

        <!-- Log4j2 — logging -->
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-core</artifactId>
            <version>2.23.1</version>
        </dependency>

        <!-- Commons-io — file utilities (copying screenshot files) -->
        <dependency>
            <groupId>commons-io</groupId>
            <artifactId>commons-io</artifactId>
            <version>2.16.1</version>
        </dependency>
    </dependencies>
```

How versions behave (so you're never trapped): Selenium releases almost monthly (it is on the 4.4x line in 2026); TestNG is on the 7.12 line. If the interviewer asks "which version?" — state the number, then add: *"in the project we keep updating to the latest stable from mvnrepository.com."* What really matters is knowing the **groupId : artifactId : version** (GAV) format.

MEMORY HOOK for the list: Selenium, TestNG, WebDriverManager, Extent, POI, Log4j, Commons → *"Some Tests Want Evidence, POI Logs Clicks."*

13.3 Maven concepts you must be able to explain

- **What Maven actually does** — 3 jobs: builds the project, downloads/manages the JARs, enforces the standard folder layout.
- **pom.xml** — "Project Object Model" — the single file where dependencies, plugins, and build settings live.
- **Where do the JARs go?** Maven downloads them from **Maven Central** into a local `.m2` folder on your machine — first build is slow, later builds reuse the cache.
- **Scope** `test` — that JAR is available only to test code; `compile` (default) — everywhere.
- **Build lifecycle (memorize the chain):** `validate → compile → test → package → verify → install → deploy`.
  MEMORY HOOK: *"Very Cool Tests Produce Very Impressive Deployments."*
- **Daily commands:** `mvn clean` (delete target/), `mvn compile`, `mvn test`, `mvn clean test`, `mvn dependency:tree` (debug JAR conflicts).
- **maven-surefire-plugin** — the plugin that finds testng.xml and runs the suite during the `test` phase. Senior line: "We configure surefire in pom.xml to point at our testng.xml, so `mvn test` runs the whole suite."
- **Why Maven instead of manual JARs?** Without Maven, every teammate and the CI server must download the same JARs by hand, and versions drift. With Maven, `mvn test` resolves everything automatically — same versions everywhere.

13.4 The CI/CD chain (say as one flow)

SAY THIS: "Code is pushed to **Git** → the **Jenkins** job triggers (on commit or nightly) → runs `mvn clean test` with testng.xml → Jenkins publishes the **Extent/Allure HTML report** → result notification goes by email/Slack."

MEMORY HOOK: *"Push → Trigger → Build → Test → Report → Notify."*

- **Extent Reports** — HTML report with pass/fail/skip, screenshots, dashboards.
- **Log4j2** — logs each step; levels TRACE < DEBUG < INFO < WARN < ERROR.
- **Selenium Grid** — run in parallel across many browsers/machines; Grid 4 runs nicely with Docker.

SECTION 14 — JAVA ESSENTIALS ASKED ALONGSIDE SELENIUM

- **OOP pillars mapped to Selenium:** Encapsulation (locators private inside page class), Inheritance (tests extend BaseTest), Polymorphism — SAY THIS: "`WebDriver driver = new ChromeDriver();` — the reference type is the interface, the object is ChromeDriver — that's runtime polymorphism, and it lets me switch to FirefoxDriver with one line."
- Overloading = same name, different parameters (compile time); Overriding = child re-defines parent's method (runtime).
- `finally` — block in try/catch that ALWAYS runs — we quit the driver there so it closes even after a crash.
- String vs StringBuilder — building strings in loops? Use StringBuilder.
- Collections: **List** (findElements returns it — duplicates allowed), **Set** (window handles — no duplicates), **Map** (test data as key→value).
- Exception handling: try/catch/finally, `throws`, `throw`; checked (compile-time, e.g. IOException) vs unchecked (runtime, e.g. NullPointerException).
- `==` compares references; `.equals()` compares content.
- Java 8 in tests: lambda in `wait.until(d -> d.findElement(...) != null)`, streams to filter lists.

SECTION 15 — CODING QUESTIONS THEY MAKE YOU WRITE LIVE

1. **Login test with POM + TestNG + DataProvider** — the #1 hands-on; practice until you can write it in 5 minutes.
2. Explicit wait until clickable, then click.
3. Switch between two windows and return to the parent.
4. Count and print all links on a page.
5. Select a dropdown value in all 3 ways.
6. Screenshot on failure using a listener (Section 9 code).
7. A login form sitting inside an iframe.
8. Excel-driven DataProvider with Apache POI (read rows → fill Object[ ][ ]).
9. SoftAssert all form validation messages.
10. IRetryAnalyzer retry class.

SECTION 16 — RAPID-FIRE ONE-LINERS (final revision)

- Selenium talks to browsers through driver binaries using the W3C protocol.
- ID is the fastest locator; XPath is the most powerful — last resort.
- Relative XPath starts with //, absolute with a single /.
- Implicit = global blanket; explicit = targeted condition; fluent = explicit with custom polling.
- presence = in DOM; visibility = rendered; clickability = visible + enabled.
- close() one window; quit() ends the whole session.
- getWindowHandle = this window; getWindowHandleS = all windows.
- Frames need switchTo; alerts need switchTo().alert(); OS dialogs need Robot.
- Actions = mouse; without perform() nothing happens.
- TestNG setup order: Suite > Test > Class > Method; teardown exact reverse.
- Without priority, tests run alphabetically.
- Hard assert halts; soft assert collects everything and speaks at assertAll().
- DataProvider = many rows in code; Parameters = one value from XML.
- NoSuch = never found; Stale = found, then the page re-rendered.
- POM is the pattern; PageFactory is one implementation of it.
- Listener: Base holds the driver, onTestFailure hears the failure, TakesScreenshot is the proof.
- CI flow: Push → Trigger → Build → Test → Report → Notify.

SECTION 17 — CLOSING ANSWER + REVISION PLAN

Q: "What is the strength of your framework?"

> "Maintainability and stability. Locators change in ONE place thanks to POM. Test data sits outside the code in Excel/properties, so coverage grows without code changes. Explicit waits and retry logic control flakiness. Extent screenshots make failure analysis fast. And Jenkins runs the whole suite nightly, so a regression is caught within a day — the framework scales from one browser to Grid parallel runs without touching the test code."

MEMORY HOOK: *"One place, one data file, one command."*

Last-day revision plan

1. Speak the 60-second framework script out loud 5 times.
2. Recite "Can Big Projects Test Until Reports Come" and draw the tree from memory.
3. Write the TestNG annotation order from memory (hotel: Suite > Test > Class > Method).
4. Revise the exceptions table — especially Stale vs NoSuch.
5. Read Section 16 twice the night before.

Walk in like the framework is YOURS — after this guide, it is.

- First item
- Second item with **bold**
- Nested item with *italic*
- Another nested item
- Third item

Java Code Example

```
function greeting(name) {
return `Hello, ${name}!`;
}
```

Python Example

```
def fibonacci(n):
if n <= 1:
    return n
else:
    a, b = 0, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b

# List comprehension example
squares = [x**2 for x in range(10)]
```

Blockquote Example

> This is a blockquote.
> It can span multiple lines.

Table Example

Feature	Description
Tables	Organized data display
Lists	Bullet points and numbers
Code	Syntax highlighted blocks

Link Example

[Visit GitHub]

Cat Image Example

